

CIRASAME Slow Control Software Manual

immediate

Rintaro Okazaki¹
okazaki.rintaro.47a@st.kyoto-u.ac.jp

and

Ken Suzuki²
ken.suzuki@rcnp.osaka-u.ac.jp

¹PhysicsDepartment, Kyoto University

²RCNP, Osaka University

Version 0.9
November 27, 2024

Contents

1	Introduction	3
2	CIRASAME	4
2.1	Overview	4
2.2	動作電圧	5
2.3	MIKUMARI 依存と Standalone	5
2.4	CITIROC	5
2.5	Bias Voltage Application	6
2.5.1	CIRASAME 毎の調整	6
2.5.2	チャンネル毎の調整	6
3	CIRASAME SlowControl Software Overview	8
3.1	HUL Common Library	8
3.1.1	Installation	8
3.1.2	設定ファイルと基本設定	8
3.1.3	set_max1932	10
3.2	CIRASAME Control Software	11
3.3	CITIROC Control Software	11
3.4	上位ソフトウェア群	11
3.4.1	インストール	11
3.4.2	設定ファイルと基本設定	12
3.4.3	commonConfigReader.py ライブラリ	13
4	CIRASAME Operation	15
4.1	バイアス電圧を印加する	15
4.1.1	ボードに共通電圧を印加する	15
4.1.2	微調整用個別バイアスを設定する	16
4.2	アナログプローブシグナル	17
4.3	プリアンプの設定	18
4.4	雑	18
4.4.1	CIRASAME ボード達が活着ているか確認する	18
4.4.2	Firmware バージョンチェック	18
5	Gain Calibration	19
5.1	Overview	19
5.2	Scaler 機能を用いた Dark Count Rate 測定	19
5.3	DCR Threshold Scan	20
5.3.1	Threshold scan raw data の取得	21
5.3.2	threshold scan raw data の整形	23
5.3.3	整形された threshold scan データから ROOT TGraph 形式への変換	23
5.3.4	Threshold scan データの解析	24
5.4	Threshold scan データ解析結果を元に CIRASAME 毎の較正を行う	24
5.5	Threshold scan データ解析結果を元にチャンネル毎の較正を行う	25
5.6	Threshold scan データ解析からしきい値を反映させる	26
5.7	較正された CIRASAME 設定を設定ファイルに反映させる	26
5.7.1	CIRASAME 設定ファイルを CIRASAME に送り反映させる	27
5.8	実際のゲイン較正の手順の例まとめ	27
6	Temperature Monitoring	29

Chapter 1

Introduction

本マニュアルは多チャンネル SiPM 読み出し・制御 ASIC である CITIROC チップ [1] を搭載した CIRASAME ボード [3] を Streaming DAQ NestDAQ[4, 5] で読み出すソフトウェアについて記述する。ソフトウェアは基本的に 2024 年 4 月 - 5 月にかけて行われた J-PARC T103 実験 [6] で開発・テストされた。次回のビームタイム等で揉まれて大きく改善されると思われるが、全体として Naming Scheme やライブラリの使用方法が consistent でない点がある。本稿では基本的に上記ビームタイム終了時のバージョンの場合を解説する。バグリポート、UI へのフィードバック・要望を是非著者までお寄せください。

説明は一般的なシステム構成を意識して記述するように心掛けたが、多少なり先述のビームタイムの環境の構成を想定しているところがある。ちなみに当時の環境は BFT (Beam Fibre Tracker) に CIRASAME ボードが 12 台、SFT (Scattered-Particle Fibre Tracker) に CIRASAME ボードが 6 台の計 18 台の CIRASAME ボードが使用されており、CIRASAME システムの合計読み出しチャンネルは 2304、CIRASAME ボードには $n=1-18$ までの番号が振られ、 $192.168.2.100+n$ という IP Address が割り当てられているという構成であった。クロック同期システムである AMANEQ システムも繋がっていた。

Chapter 2

CIRASAME

2.1 Overview

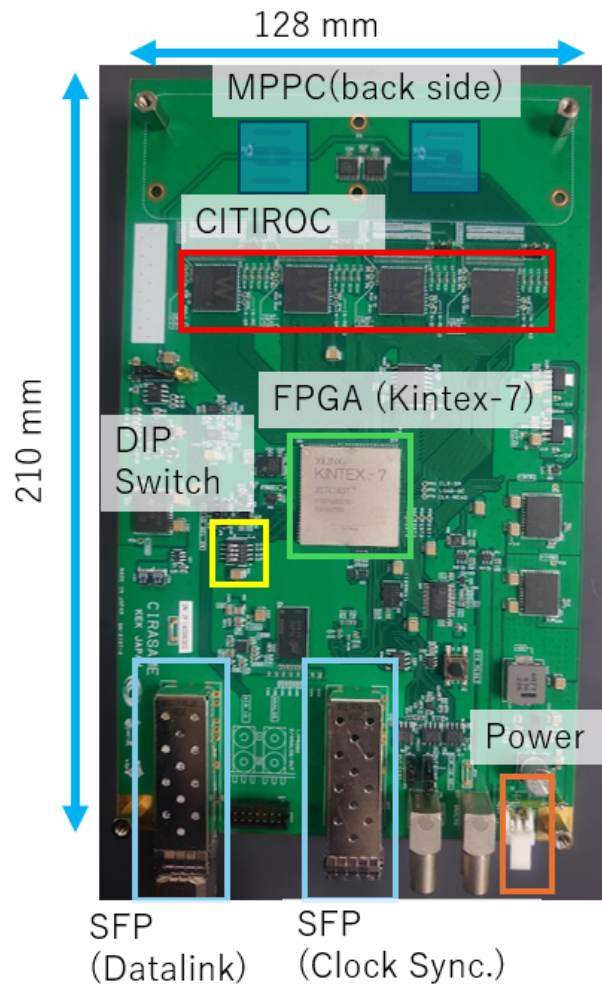


Figure 2.1: CIRASAME の画像

CIRASAME は SiPM の読み出し用 ASIC である CITIROC を用いた 8×8 チャンネル MPPC 読み出し用フロントエンド回路である。CITIROC は 1 枚当たり 32 チャンネルの読み出しが可能であり、これを 4 枚用いることによって 2 枚の 8×8 チャンネル MPPC 計 128 チャンネル分を CIRASAME1 台で読み出せる。

CITIROC に入った信号は内部で信号増幅+波形整形+ Discriminator を通過し、FPGA へと出力される。その後、図 2.1 の左下のポートより DAQ 用 PC へとデータが送信される。

Streaming DAQ によって信号を読み出す以外にも Clock Sync と電源の間にある LEMO 端子から、Citiroc 内部の Probe からの信号や NIM 信号を見ることが可能。

2.2 動作電圧

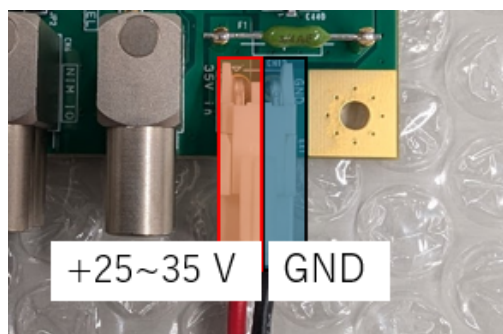


Figure 2.2: CIRASAME への電力供給

CIRASAME は図 2.1 右下の電源用端子にコードを接続し、20-35V の電圧をかけることによって起動する。また、端子の極性は図 2.1 で見て左側が陽極、右側が陰極 (GND) になっている。

2.3 MIKUMARI 依存と Standalone

実際の DAQ ではすべての CIRASAME は、MIKUMARI からのクロックを参照する必要がある。そのため、基本的には図 2.1 の Clock Sync. の部分を MIKUMARI に接続しなくてはならない。

一方で、Gain Calibration 時では CIRASAME 全体でクロックを統一する必要がないため、MIKUMARI 未接続でも動作させることが可能である (もちろん MIKUMARI が接続されていても問題がないが、その際は MIKUMARI 依存モードで運転させる)。この時に CIRASAME が動作するためには CIRASAME を Standalone モードで起動する必要がある。

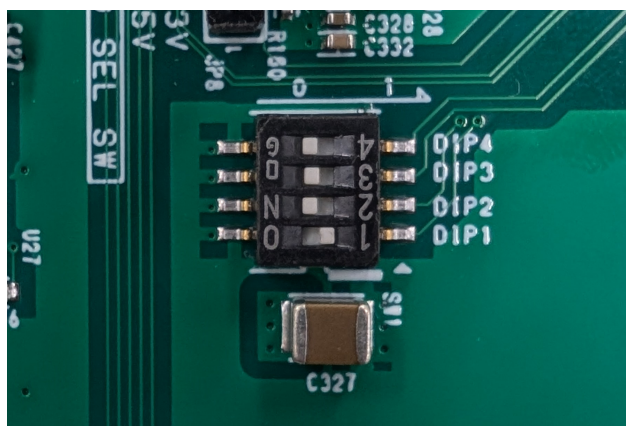


Figure 2.3: CIRASAME 上の DIP スイッチ拡大図

MIKUMARI 依存モードと Standalone モードの切り替えは図 2.1 の DIP スイッチで行われる。図 2.3 には 4 つのスイッチがあり、DIP3 のスイッチが 0 にあるときには MIKUMARI 依存モード、1 の時に Standalone になっている。

2.4 CITIROC

CITIROC は図 2.4 のように Ch. 0 から Ch. 31 までの計 32 チャンネルの信号を処理できる ASIC である。各信号は個別に PreAmp により増幅され、Shaper(fsh) を通して整形、Discriminator (10-bit) を通り、Time trigger として信号が FPGA へと送信される。

InputDAC をはじめとして PreAmp を HG, LG のどちらにするか、またその増倍率を変化させるには slow-control 時に CITIROC に送るレジスタ値の変更を行う。レジスタ値の操作・反映については章 5 を参照。

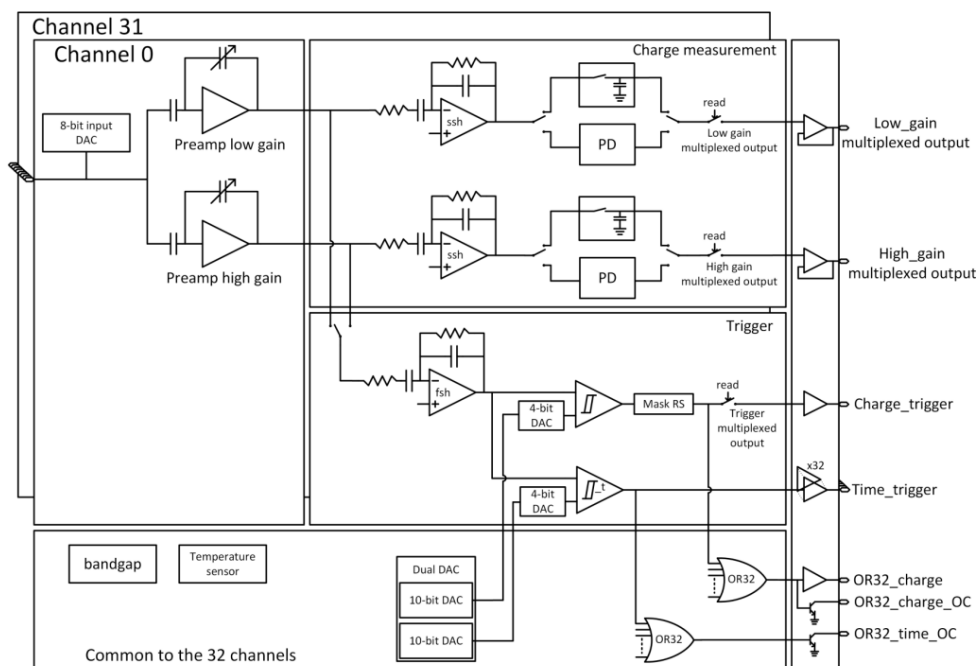


Figure 2.4: CITIROC の回路図

2.5 Bias Voltage Application

CIRASAME および CITIROC には SiPM のバイアス電圧を調整するためのパラメータが2つ存在している。以下に記述されている内容には、3章と重複することもあるが、重要事項のため先に記載している。

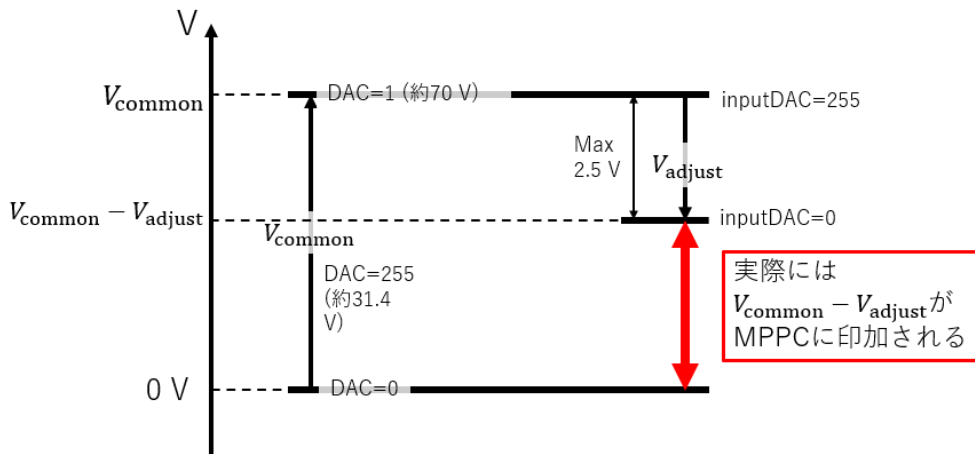


Figure 2.5: 実際に SiPM に印加される電圧は CIRASAME ボードの機能で提供される電圧 V_{common} (40-70 V) 及び CITIROC チップの提供するチャンネルごとの微調整用電圧 V_{adjust} (0-2.5 V) の2弾構成になっている。XXX

2.5.1 CIRASAME 毎の調整

1つ目は CIRASAME の機能として、「その CIRASAME に搭載している MPPC の全チャンネルに印加される」電圧である V_{common} である。こちらはチャンネルごとに個別に調整することはできず、大まかなゲイン合わせのためのものである。

2.5.2 チャンネル毎の調整

2つ目は CITIROC の機能として、「チャンネルごとに微調整が可能」な V_{adjust} である。これは CITIROC の InputDAC パラメータによって調整される。 V_{adjust} はチャンネル別に調整が可能なパラメータである一方、動かせる電圧幅は 2.5 V しかない。特にこの機能は V_{common} とは逆方向に印加されるため、注意が必要。

図 2.5 において、 V_{adjust} を調整するための inputDAC は 0-255 の値をとる。InputDAC が 0 の時に最大値である 2.5 V を出力し、数値を大きくするにしたがって V_{adjust} は小さくなり、最終的に inputDAC を 255 にした際に 0 V になる。InputDAC を 1 大きくした際の出力の下がり幅は線形であり、 $2.5/256 \approx 0.01 \text{ V}$ ずつ低下していく。

Chapter 3

CIRASAME SlowControl Software Overview

ここでは CIRASAME SlowControl Software の概観及び基本的なセットアップについて述べる。CIRASAME SlowControl Software は PC と CIRASAME との communication を行う下位の基礎ライブラリ (hul-common-lib、基本的に C++ で書かれている) およびそれらを用いて視覚化・解析等種々のタスクをこなす上位のスクリプト群 (Shell Script, Python Script) からなる。

設定ファイルも上位下位ソフトウェアで異なっている。下位ライブラリは CIRASAME ボード毎の複数の yaml 設定ファイルを使用する。yaml ファイルの人間による可読性は比較的高いが、定義されている設定項目、設定値を知っている必要があり、また多数のファイルに分散していることから上位ソフトウェアでは主にスプレッドシートを用いている。これらの上位下位設定ファイルの変換機能も上位ソフトウェア群が提供する。

Gain Calibration などの特定のタスクに関するスクリプトの説明は次章以降の相当する項で説明する

3.1 HUL Common Library

CIRASAME 用に用いられている slow controll ソフトウェアはその前身となる HUL ボード [7] で開発されたものがベースとなっておりの名前を引き継いでいる。

3.1.1 Installation

レポジトリはここ (<https://github.com/spadi-alliance/hul-common-lib>)。ファイルは `~/cirasame/hul-common-lib/` ディレクトリ以下にインストールすることを推奨する。本マニュアルでは以下これを前提として記述する。

3.1.2 設定ファイルと基本設定

CIRASAME ボード上のレジスタへデータ送信を行う hul-common-lib は基本的に `~/cirasame/calib/config` 以下に保存された yaml ファイルを参照する。CIRASAME ボードは 1 から sequential に番号を振って区別する。hul-common-lib では 3 digits 0-padding で cirasame001 等として参照される。

`~/cirasame/calib/config` 以下には全 CIRASAME を対象としたファイル (ipmap.yaml, commonpath.yaml, thresholdscan_range.yaml) と cirasame001 といったように各 CIRASAME 毎のディレクトリがある。cirasame 毎のディレクトリには DiscriMask.yaml, InputDAC.yaml, RegisterValue.yaml, settings.yaml ファイルがある。

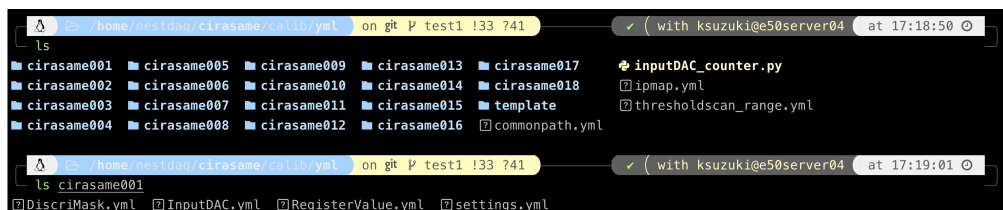


Figure 3.1: `~/cirasame/calib/config` ディレクトリ及び `~/cirasame/calib/config/cirasame001` ディレクトリ内のファイル構成例。

全 CIRASAME ボード共通の設定ファイル `~/cirasame/calib/config` ディレクトリに複数の設定ファイルが置かれている。これらは章 3.4.3 に後述するように現状上位ソフトウェア群のみによって利用されている¹。

CIRASAME 毎の設定ファイル 使用している CIRASAME ボード毎に `~/cirasame/calib/config/cirasame001` といったディレクトリがあり、各々に以下の 4 つの設定ファイルがある。それぞれ `DiscriMask.yml`、`InputDAC.yml`、`RegisterValue.yml`、`settings.yml` である。以下に各ファイルについて説明する。

DiscriMask.yml 各チャンネルごとに読み出す読み出さないを設定できる。とりうる値は 1bit 0-1 である。Mask ファイルなので読み出す場合は 0、読み出さない場合は 1 を設定する。マスクされたチャンネルにくるシグナルは無視される。FEE のリソースを食いつぶさないために不要なチャンネル、ノイジーなチャンネルはマスクする。

```
Δ ~/cirasame/calib/config/cirasame001 on gh P test1 !33 741 ✓ ( with ksuzuki@e50server04 at 12:38:49 ○
wc DiscriMask.yml
35 68 228 DiscriMask.yml

Δ ~/cirasame/calib/config/cirasame001 on gh P test1 !33 741 ✓ ( with ksuzuki@e50server04 at 12:39:02 ○
head -10 DiscriMask.yml | cat
STDIN
1 ---
2 CITIROC1:
3 Discriminator Mask:
4 - 0
5 - 0
6 - 0
7 - 0
8 - 0
9 - 0
10 - 0
```

Figure 3.2: `~/cirasame/calib/config/cirasame001/DiscriMask.yml` ファイルの例。

InputDAC.yml `InputDAC.yml` は各チャンネルごとの `InputDAC` の値が記述される。とりうる値は 8bit(0-255) で 0 ~ 2.5V の範囲で V_{adjust} を出力する。実際に印加される電圧は (V_{common} と (V_{adjust} の差 ($V_{\text{bias}} = V_{\text{common}} - V_{\text{adjust}}$) となることに留意すること。図 2.5 参照のこと。図 3.3 に `InputDAC.yml` の記述例を示す。ゲイン較正の前では上下の振幅を均等に確保するため中間値 127 に設定しておくことが推奨される。

```
Δ ~/cirasame/calib/config/cirasame001 on gh P test1 !33 741 ✓ ( with ksuzuki@e50server04 at 12:40:47 ○
wc InputDAC.yml
136 272 1140 InputDAC.yml

Δ ~/cirasame/calib/config/cirasame001 on gh P test1 !33 741 ✓ ( with ksuzuki@e50server04 at 12:40:58 ○
head -10 DiscriMask.yml | cat
STDIN
1 ---
2 CITIROC1:
3 Discriminator Mask:
4 - 0
5 - 0
6 - 0
7 - 0
8 - 0
9 - 0
10 - 0
```

Figure 3.3: `~/cirasame/calib/config/cirasame001/InputDAC.yml` ファイルの例。

RegisterValue.yml `RegisterValue.yml` は CIRASAME ボード設定レジスターの内容を記述する。内部はボード全体の設定及び CITIROC ASIC 毎の設定からなる。設定項目及び設定可能な値は `~/cirasame/CitirocControlSoft/src/configLoader.cc` に定義されている。

settings.yml `settings.yml` は主にゲイン調整をする際のスキャン範囲を指定するパラメータが格納されている。スキャン範囲を広げたり、スキャン時の刻み幅を変更したい際にはこのファイルを編集する。現状のプログラムではスキャン範囲の指定にのみ使用している。Listing2 に書かれている `DAC.scan` がスキャン範囲を決定するパラメータ群である。`start` はスキャン開始時の `thresholdDAC` の値。`gap` はスキャンの `thresholdDAC` の刻み幅で、`steps` の回数 `gap` 分ずつ増加しつつダークカウントレートが計測される。

¹直接 `CitirocControlSoft` 内の `femcitiroc.control` を実行する場合は引数として対応する CIRASAME の `config` フォルダ内のファイルを指定しなくてはならない。しなかった場合は `CitirocControlSoft` 内にある `default_register` の設定が参照される。

```

Analog Mux: Mux_HG
CITIROC1:
  8-bit DAC reference: 2.5V
  DAC1 code: 1000
  DAC2 code: 150
  EN_LG_Pdet: Enable
  EN_LG_T&H(Widlar SCA): Enable
  EN_Low_Gain_PA: Enable
  EN_Probe OTAq: Enable
  EN_High_Gain_Slow Shaper: Enable
  EN_Low_Gain_Slow Shaper: Enable
  Fast Shaper on LG: HG
  LG PA bias: weakbias
  PreAMP: HGcf_700fF
  SCA bias: highbias
  Sel Trig Ext PSC: Enable
  Time Constant HG Shaper: 50.0ns
  Time Constant LG Shaper: 50.0ns
  bypass PSC: Enable
CITIROC2:
  8-bit DAC reference: 2.5V
  .....
CITIROC3:
  8-bit DAC reference: 2.5V
  .....
CITIROC4:
  8-bit DAC reference: 2.5V
  .....
  bypass PSC: Enable
High Gain Channel: 0
Probe: Out_fs
Probe Channel: 0
Probe Mux: 1

```

Listing 1: ~/cirasame/calib/config/RegisterValue.yml ファイルの例。

```

CITIROC_path : ../CitirocControlSoft/bin
HUL_path     : ../hul-common-lib/install/bin
YAML_path    : ../CIRASAME_calib/yaml_files/cirasame001
CIRASAME_ip  : 192.168.2.101
DAC_scan:
  start : 150
  gap   : 1
  steps : 450
FILE_name : aaa

```

Listing 2: ~/cirasame/calib/config/settings.yml ファイルの例。

3.1.3 set_max1932

V_{common} を設定する際に使用する。

```

~/cirasame/hul-common-lib/install/bin/set_max1932 <IP Address> <bias value>
<bias value>:
0       : Gives almost 0V as HV (= OFF)
1-255   : Gives HV as following equation:

```

3.2 CIRASAME Control Software

`~/cirasame/CirasameControlSoft/`以下にインストールされている。現状使用していない。

3.3 CITIROC Control Software

`~/cirasame/CitirocControlSoft/`以下にインストールされている。ここ以下の `bin/femcitiroc_control` が主に使用される binary。

```
~/cirasame/hul-common-lib/install/bin/femcitiroc_control
Usage
  femcitiroc_control [options...]
Options:
  -ip=xxx.yyy.zzz.aaa (Must)
  -yaml=file-path
    If -yaml=auto is designated, this program refers
    the register files in ${HOME}/vme-easiroc-registers,
    which are labeled by the ip address.
    i.e., RegisterValue_aaa.yml, InputDAC_aaa.yml, PedSup_aaa.yml
  -sc
    Send slow control registers
  -read
    Send read slow control registers
  -probe
    Send probe registers
  -all
    Equal to -sc -read -probe
  -probe-off
    Reset probe registers
  -q
    Quiet. Hide all the message.
```

この -yaml オプションは `RegisteValue.yml`, `InputDAC.yml`, `DiscrimMask.yml` の 3 つをそれぞれ `-yaml RegisteValue.yml -yaml InputDAC.yml -yaml DiscrimMask.yml` のように指定しなくてはならない。

3.4 上位ソフトウェア群

ここまでで解説した下位ソフトウェア群に主に有用な UI を提供し特定のタスクをこなすソフトウェア群で主にスクリプト言語で記述されている。

3.4.1 インストール

Cloud 上のレポジトリは現状特に用意されていない。よりよいソフトの配布方法は今後検討するが、とりあえず近日中に zip を用意する。`~/cirasame/calib/`以下に展開されることが想定されている。`~/cirasame/calib/`ディレクトリは git で管理されている。ディレクトリのツリー構造を図に示す。具体的な説明は以下の関連する項目で行う。

以下の環境設定が推奨される。

PATH 環境変数

環境変数 PATH に以下の path を足す。

- `/cirasame/bin:`
- `/cirasame/calib/bin:`
- `/cirasame/calib/ana`
- `/cirasame/hul-common-lib/install/bin`
- `/cirasame/CitirocControlSoft/bin`



Figure 3.4: `~/cirasame/calib/`以下のディレクトリの概要。リストが長くなりすぎないように `ana/log`, `ana/out`, `ana/pic`, `ana/root`, `config`, `data`, `raw`, `xls` ディレクトリの中身は適当に短くしてある。整理しきれていない、すでに使われていないと思われるスクリプトなどもあるので注意。

Root Macro PATH

`.rootrc` に以下の path を足す。

- `~/cirasame/calib/ana`

例:

```
Unix.*.Root.MacroPath:      .:$(HOME)/cirasame/calib/ana:$(ROOTSYS)/macros
```

3.4.2 設定ファイルと基本設定

CIRASAME ボード Slow Control 用の設定内容は多くのパラメーターが関与するため、可読性と変更・管理の用意さを考慮してスプレッドシート形式でまとめている。このファイルをスクリプト等で直接読み書きする。共同実験者との並行作業が用意になるため Cloud 上の Google Spreadsheet をマスターとして使用している。しかし、Python スクリプト等で PC から直接読み書きをする API は有料な為、現在のところ Google Spreadsheet 上で変更した内容をローカルに保存してそのファイルに対して読み書きを行うということを行っている。現状では書き込みされたファイルの Cloud へのアップロード機能は無く、手動で行うことになっている。

設定ファイルの作成方法

Slow Control 用の設定内容は大きく分けて以下の 2 パートに分かれている。

1. 使用する CIRASAME の情報まとめシート

2. 各 CIRASAME の V_{adjust} 調整用シート

これら2種類は別のシートとして同一のファイル内に記述されている必要がある。したがって、Google Spreadsheet からローカルにファイルをエクスポートする際には.xlsx の形式で行う。

3.4.3 commonConfigReader.py ライブラリ

ソフトウェア群全体で、「同じ情報を2箇所に書かない」ルールを進めるため、複数のファイルに冗長にハードコードされていた情報をまとめる作業が進められている。その目的のために`~/calib/config/`以下に全 CIRASAME ボード共通の設定ファイルが用意されている。この情報は主に `commonConfigReader.py` ライブラリによってアクセスされ、このライブラリは各 python スクリプトによって利用される。が現状移行は完全に終了していない。

ipmap.yml `ipmap.yml` は使用する CIRASAME ボードの番号及び該当ボードの IP Address がリストとして記述されている。本例では `192.168.2.0/24` サブネットが使用されている。`commonConfigReader.py` ライブラリも参照する。現行の操作手順では使用されていないが、完全に移行が完了するまで保管している状態。

```
# map of cirasame IP address
cirasameIP:
  1 : 192.168.2.101
  2 : 192.168.2.102
  3 : 192.168.2.103
  4 : 192.168.2.104
  ....
  ....
```

Listing 3: `~/cirasame/calib/config/ipmap.yml` ファイル例。

commonpath.yml の記述 `commonConfigReader.py` ライブラリも参照する。こちらも現在は使用していないが、保管中。²

```
# PATH used for all CIRASAME
hulcommonlib : ~/cirasame/hul-common-lib/install/bin
citirocccontrolsoft : ~/cirasame/CitirocControlSoft/bin
config : ~/cirasame/calib/config
```

Listing 4: `~/cirasame/calib/config/commonpath.yml` ファイルの例。

thresholdscan-range.yml の記述 `commonConfigReader.py` ライブラリも参照する。`Thresholdscan` の範囲を変更する場合はここの数値を変更する。

²歴史的経緯により、最初は `cirasame` 別に管理していたことを言及

A1	CITIROC#	Channel#	Bias	Individual
2	1	1	165	
3	1	2	165	
4	1	3	165	
5	1	4	165	
6	1	5	165	
7	1	6	165	
8	1	7	165	
9	1	8	165	
10	1	9	165	
11	1	10	165	
12	1	11	165	
13	1	12	165	
14	1	13	165	
15	1	14	165	
16	1	15	165	
17	1	16	165	
18	1	17	165	
19	1	18	165	
20	1	19	165	
21	1	20	165	
22	1	21	165	
23	1	22	165	
24	1	23	165	

Figure 3.5: cirasame.settings.xlsx の例。複数ボードで組まれシステム全体に関する設定内容 (上) と各ボード毎の設定内容 (下) からなる。これらがタブとして一つのスプレッドシートに纏めてある。**(ここに CIRASAME ボードの個体認識が ip address 基準で行われていて、該当 IP を消すとスキップすることを記述する)

```

$ /home/hesdaq/cirasame/calib/yaml on git P test1 !33 ?41
cat thresholdscan_range.yml

File: thresholdscan_range.yml

1 # Range of threshold scan
2 start : 150
3 end   : 450
4 step  : 1

```

Figure 3.6: ~/cirasame/calib/config/thresholdscan-range.yml ファイルの例。

Chapter 4

CIRASAME Operation

CIRASAME Slow Control Software を用いて CIRASAME ボードのレジスタの書き換え、特に SiPM にバイアス電圧を掛ける手順を解説する。

4.1 バイアス電圧を印加する

2 章および図 2.5 で説明したように SiPM への印加電圧は CIRASAME ボードが提供してボード全体に一定の共通電圧を掛ける (V_{common}) と CITIROC が提供してチャンネルごとに 0-2.5 V の微調整用電圧 (V_{adjust}) を掛ける 2 段階構成になっている。電圧を掛ける作業は各々別となる。以下に順番に説明する。

4.1.1 ボードに共通電圧を印加する

手動で特定の CIRASAME ボードに電圧を印加する

CIRASAME の機能を用いてボード上の全 SiPM に共通のバイアス電圧 V_{common} を手動で与える場合、`bias_on.sh` / `bias_off.sh` を用いる。内部で `set_max1932` コマンドが呼ばれる。電圧は DAC 値を使う。電圧 V で指定できる改良はまだできていない。現在掛かっているバイアス設定を読むことはできない¹。

```
~/cirasame/bin/bias_on.sh <CIRASAME ID (1-18)> <bias value>
<bias value>:
0          : Gives almost 0V as HV (= OFF)
1-255     : Gives HV as following equation:
```

DAC 値と実際の電圧の関係はおおよそ

$$V_{\text{common}} = -0.114 \times \langle \text{bias value} \rangle + 70.556 \quad (4.1)$$

となっている²。例として、Hamamatsu MPPC S11XXXX, 推奨オペレーション電圧 54 V³ の場合はおおよそ 120 程度になる。実際には V_{adjust} の分印加電圧が下がるので (図 2.5 参照) それを考慮する。推奨のように $V_{\text{adjust}} = 127$ の場合は 1.25 V 低下するのでそれに相当する DAC 値 10 を引いて掛ける。つまりこの場合の V_{common} 初期値は 110 程度を推奨する。逆操作として `bias_off.sh` で印加電圧を 0 にする。

```
bias_on.sh <CIRASAME ID> 0
```

と等価。

手動で複数の CIRASAME ボードに一齐に電圧 (V_{common}) を印加する

回路構成に合わせ変更する必要あり。

`bft_bias_on.sh` / `sft_bias_on.sh`

```
~/cirasame/bin/[s|b]ft_bias_on.sh <bias value>
```

¹最後に設定したバイアスを記憶させることはできそうだが、現段階では用意されていない。

²本ソフトウェアではこの値を使用しているが、この関係はボードによって正確には異なることが考えられる。実用面では現状問題が無いように思われるが、理解の上使用してほしい。

³2024/4-6 のビームタイムでは推奨電圧 53.5-54.1 V の MPPC を使用した。

設定ファイルを用いて複数の CIRASAME ボードに一斉に電圧 (V_{common}) を印加する

apply_bias.py コマンドを利用する。

用法 上記のように command line argument としてバイアス値を指定するのではなく、バイアス情報が正しく記述された設定ファイル (.xlsx) から情報を読み取りバイアス電圧を設定する場合、apply_bias.py を用いる。本コマンドは通常テストモードで起動する。一度コマンドを起動し出力から操作が問題ないことを確認したうえで [-x] オプションを付けて実際に印加する。

```
apply_bias.py [-h] -n <SPREADSHEET FILENAME> [-o OFFSET] [-x]
```

optional arguments:

```
-h, --help            show this help message and exit
-n NAME, --name NAME  excel file name
-o OFFSET, --offset OFFSET
                        If you want to give some common offsets to bias, input an integer.
-x                    Add this option when you execute
```

読み込みファイル

- `~/cirasame/calib/xlsx/<file name>.xlsx`

出力ファイル 無し

例

```
apply_bias.py -n <path to your xlsx file> -x
```

4.1.2 微調整用個別バイアスを設定する

チャンネル個別にバイアス電圧を調整する機能を持つ V_{adjust} を設定する。こちらは基本的に大量のチャンネルを設定することになるので特定のチャンネル単体の設定を変える機能は現在用意されていない。作る予定はあり。 V_{adjust} の値はスプレッドシートで管理している。この値を下位ソフトが使用する yaml ファイルに書き出し、そこから CIRASAME ボード上のレジスターに転送する 2 ステップの作業になる。

スプレッドシートの内容を yaml に書き出す

xls2yaml.py を用いる。これは ~/cirasame/bin にある。基本的にスプレッドシートの内容全体を yaml ファイルに書き出す機能となる。特定の設定のみを書き出す機能は現状実装されていない。

用法 この Python script は

```
python3 xls2yaml.py
```

という invoke が必要⁴。デフォルトではドライモードで起動され実行される内容が STDOUT に出力される。それを確認の後 -x オプションを付けて実行することで実際の書き込みを行う。⁵ 特定のチャンネルの V_{adjust} の yaml ファイル設定をインタラクティブに変更するコマンドを用意する構想があるがまだ実装されていない。-m オプションでリモート (クラウド) とローカルを指定できる様になっているが、上記の理由で -m オプション無しで起動するとデフォルトで local mode になる。

```
python3 xls2yaml.py [-h] [-m {remote,local}] [-x] [-l {DEBUG,INFO,WARNING,ERROR,CRITICAL}]
                    file_path_or_id
```

positional arguments:

```
file_path_or_id      Path to the local Excel file or ID of the Excel file on
                        Google Drive
```

optional arguments:

```
-h, --help            show this help message and exit
-m {remote,local}, --mode {remote,local}
```

⁴TODO List に入っている

⁵required as .xlsx file TODO: subscribe to Google API so that the script can directly read and write the spreadsheet online. Process Excel file and invoke C++ binary script.

```

-x, --execute          Mode to run (remote or local) Only local mode is currently supported
                        Execute the script (default: dry run)
-l {DEBUG,INFO,WARNING,ERROR,CRITICAL}, --loglevel {DEBUG,INFO,WARNING,ERROR,CRITICAL}
                        Set the logging level

```

読み込みファイル

- `~/cirasame/calib/cirasame_setting.xlsx`

出力ファイル

- `xlsx` 内で指定された `RegisterValue.yml` ファイル。通常 `~/cirasame/calib/config/cirasame<n>/RegisterValue.yml` (`n=1-18`)
- `xlsx` 内で指定された `InputDAC.yml` ファイル。通常 `~/cirasame/calib/config/cirasame<n>/InputDAC.yml` (`n=1-18`)

例

```
~/cirasame> python3 bin/xls2yml.py calib/xlsx/crsm_setting_adjustertest.xlsx -x
```

yml ファイルの設定内容を CIRASAME ボードに転送する

`register2cirasame.sh` を用いる。歴史的に `slowcontrol.sh` という名前の Symbolic link がある。 `yml2cirasame.sh` の方が分かり易い可能性もあるので Symbolic link を用意してある。レジスター設定ファイルの情報を CIRASAME に転送する。レジスター情報が全てアップロードされるので他の設定も変更されるが、 V_{adjust} を変更する際もこちらを利用する。

用法

```
register2cirasame.sh <cirasame number>
cirasame number: CIRASAME 番号
```

読み込みファイル

- `~/cirasame/calib/config/cirasame<cirasame number>/RegisterValue.yml`
- `~/cirasame/calib/config/cirasame<cirasame number>/InputDAC.yml`
- `~/cirasame/calib/config/cirasame<cirasame number>/DiscrimMask.yml`

出力ファイル 無し

例

```
register2cirasame.sh 1
```

4.2 アナログプローブシグナル

CITIROC の機能として、指定した点のシグナルを出力するプローブ機能が備わっている。どの点の信号を読み出すかは `RegisterValue.yml` の `Probe`, `Probe Channel`, `Probe Mux` を書き換えることで指定できる。 `Probe` では、図 2.4 上のどの点での信号を読み出すのかを指定する。指定できる点は以下の 6 点である。

- `None`: プローブ無し
- `Out_PA_HG`: High-gain の PreAmp
- `Out_PA_LG`: Low-gain の PreAmp
- `Out_ssh_HG`: High-gain の slow shaper
- `Out_ssh_LG`: Low-gain の slow shaper
- `Out_fs`: Discriminator 前の fast shaper

Probe Mux ではどの CITIROC から読み出すかを 1-4 で指定する。さらにその中のどのチャンネルを読み出すかを Probe Channel に 0-31 の数値を入れて指定する。

また、この設定を反映させる際には CitirocControlSoft 内の femcitiroc を実行する際に -probe オプションをつけなくてはならない。プローブのオン・オフは /cirasame/bin の中に probe_on.sh, probe_off.sh としてコマンドを用意している。

用法

```
probe_on.sh <cirasame number>
probe_off.sh <cirasame number>
cirasame number: CIRASAME 番号
```

4.3 プリアンプの設定

プリアンプは MPPC の信号を増幅するのに用いられる (図 2.4 参照)。プリアンプには High-gain と Low-gain の 2 種類があり、それぞれのプリアンプは RegisterValue.yml の PreAmp の項目を操作することで増倍率を変更することができる。どちらのプリアンプを使用するか選択するには Fast Shaper on LG を HG にすれば High-gain, LG にすれば Low-gain が採用される。

使用可能な値は結構多いため、CitirocControlSoft/src 内の configLoader.cc をご確認ください。

4.4 雑

4.4.1 CIRASAME ボード達が生きているか確認する

用法 ping コマンドを使って複数の CIRASAME ボードとのコミュニケーションが取れるか確認する。CIRASAME は 192.168.2.xxx の IP アドレスを持っていることを仮定している。

```
Usage: checkalive.sh start_ip end_ip
```

読み込みファイル 無し

出力ファイル 無し

例

```
checkalive.sh 101 118
```

4.4.2 Firmware バージョンチェック

Chapter 5

Gain Calibration

5.1 Overview

ゲインの較正は V_{common} を用いた CIRASAME ボード毎の調整と V_{adjust} を用いたチャンネルごとの調整の 2 段階で行われる。

1. 全ての channel に一定の V_{bias} がかかっている初期状態 (図 5.1 の最左の状態) から CIRASAME ボード毎に V_{common} の調整を行った状態 (図 5.1 の左から 2 番目の状態) に調整する。
2. そこから更に Channel 毎に V_{bias} を調整した状態 (図 5.1 の右から 2 番目の状態) に調整する。

現状では 2 段階目と同じ調整をイタレートすることでゲインがより精度良く揃う結果が得られている為、実際には 3 ステップが推奨される。これは今後のソフトウェアの向上で解消する可能性がある。

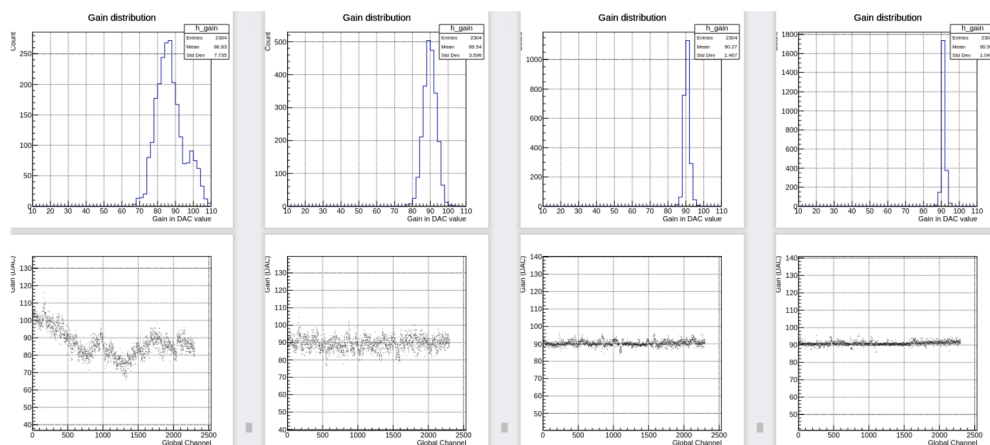


Figure 5.1: ゲイン調整各ステップでのゲイン分布ヒストグラム (上) とゲイン vs Global Channel の図 (下)。各々左から順番に 1) 全ての channel に一定の V_{bias} がかかっている初期状態 2) CIRASAME ボード毎に V_{common} の調整を行った状態 3) Channel 毎に V_{bias} を調整した状態 4) 2 から 3 のステップをイタレートした状態。ゲイン分布の RMS は各々 7.3, 3.6, 1.5, 1.0 で、最終的に 1.1% の均一度が得られている。

5.2 Scaler 機能を用いた Dark Count Rate 測定

CIRASAME にはスケーラー機能が搭載されており、DAQ を介さずに SiPM の信号が discriminator の threshold を超えた回数を測定することができる。この機能を用いて SiPM の Dark Count Rate (以後 DCR) の測定を行う。基本的なスケーラーの読み取り機能は hul-common-lib の read_scr として実装されている。

これによって指定した CIRASAME の全チャンネルのスケーラー情報が入ったファイルが出力される。しかし、このファイルの数値はバイナリで書かれてしまっているため、読み取るには od コマンドなどを使用して、10 進数に直さなくてはならない。¹

すると、Listing 7 ようなデータが見えるようになる。

¹書き方が変。推敲が必要と思われる。

```
~/cirasame/hul-common-lib/install/bin/read_scr <IP address> <path to savefile>
```

Listing 5: read_scr を用いたスケーラーの読み出し方

```
od -Ad -td -v <path to savefile>
```

Listing 6: スケーラーの出力情報を 10 進数で表示

0000000	64218	12194082	1914	0
0000016	0	0	0	0
0000032	0	0	0	0
0000048	0	0	0	0
0000064	0	0	1823116	1829434
0000080	1802661	732349	1689500	639165
0000096	1714328	568408	1675130	519771
0000112	1704686	831911	1696742	1609683
.....				
0000544	10416800	10805496	11119828	10570195
0000560	10681096	10993608	10574560	10924723
0000576	11601855	11339932		
0000584				

Listing 7: 10 進数へ変換後のスケーラーデータ (CIRASAME のファームウェアが変更されたときはこのフォーマットが変更されていることが偶にあるため注意！)

左 1 列はレジスタのアドレスである。それを除いた左上から 19 番目の数字 (Listing 7 では 1823116 の値) からが Ch. 1 からのスケーラー値になっている。

また、スケーラーの値をリセットすることも可能である。先ほどの read_scr のようにシンプルなコマンドではなく、write_register を使用してレジスタのアドレスを直接指定して記入していくことになる。

```
~/cirasame/hul-common-lib/install/bin/write_register <IP address> 0x80000000 0x1 1
```

Listing 8: write_register を用いて全チャンネルの scaler 値を 0 にする

以上のコマンドを用いて、

1. write_register でスケーラーのカウントをリセット
2. 1 秒待つ
3. read_scr でスケーラーの情報を取得

の順番で実行することによってとある threshold での DCR の取得が可能になる。

5.3 DCR Threshold Scan

SiPM のゲイン測定には複数の方法が考えられるが、本ソフトウェアでは 5.2 章で説明した DCR を測定する手法を CITIROC 内の discriminator の threshold を変えつつスキャンする手法 (Threshold Scan) を用いる。SiPM は 1 p.e. 毎の波高の均一性が高く、discriminator threshold を徐々に上げていくと n p.e. 波高と n+1 p.e. 波高の間の DCR がフラットな領域と、ちょうど n+1 p.e. 波高に相当する辺りで DCR が大きく変化する

(Transition) 領域が交互に現れる。DCR を縦軸対数表示、Threshold を横軸に plot すると階段状の図が得られる。例を Fig. 5.2 に示す。隣接する Transition 領域の間隔を測ることによって Gain を決定できる。

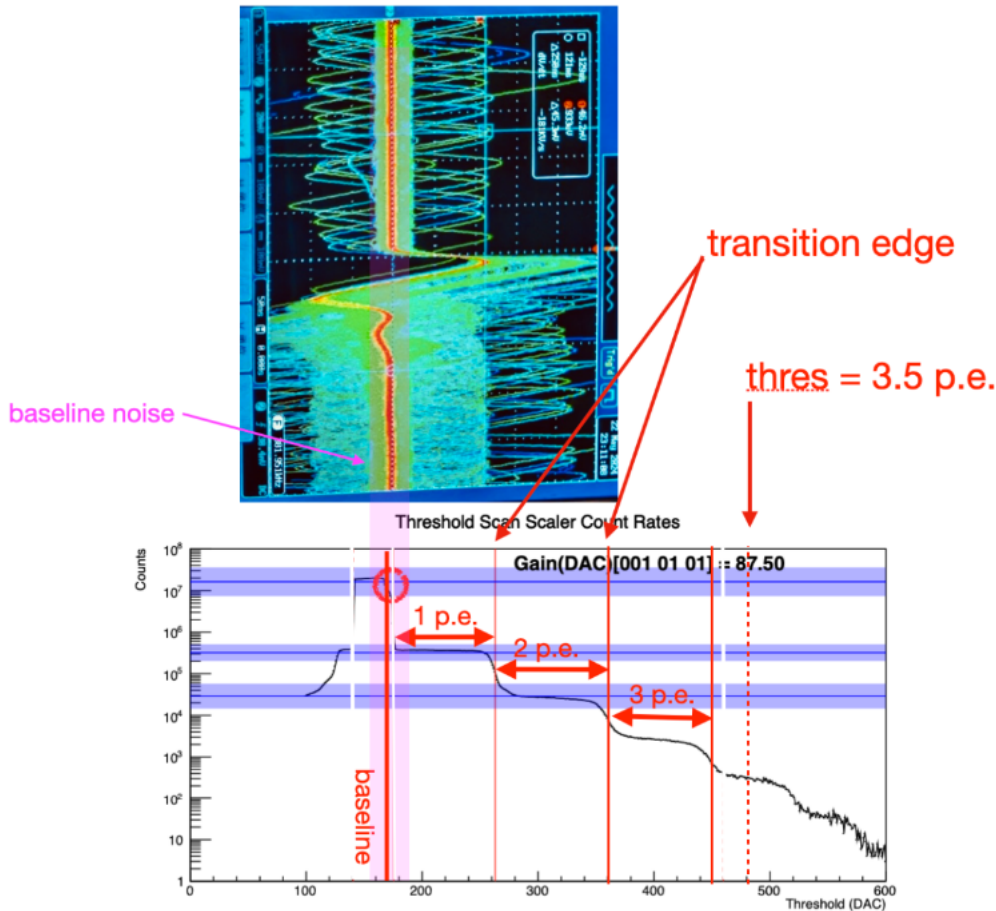


Figure 5.2: Threshold scan の例 (下) 及び SiPM の波高分布 (上) の対応。上図は SiPM のオシロスコープの出力を Persistency On で取得したものを 90 度回転させてある。下図、縦軸は DCR [Hz]、横軸は CITIROC Discriminator のしきい値の DAC 値。しきい値が p.e. 毎の波高を超えるごとに階段状の構造が現れる。この例では 5 p.e. 程度のプラトーまで見えている。DCR 測定時間は各々 1 秒。本例の DAC と実際のしきい値の関係は $V = -0.114 \times \text{DAC} + 70.567$ 。

5.3.1 Threshold scan raw data の取得

本測定はビームが出ていない状態で行うのが望ましい。しかし 1 MHz 程度のビームが出ていても 1-3 p.e. 程度までは DCR の方が大きい影響は小さい。これを discriminator の threshold 値を変更しながら scan(Threshold scan) を行う。

一つの CIRASAME ボードのスキャンを行う

一つの CIRASAME ボードのスキャンを行う際は python script thresholdscan_single.py を用いる。スキャンの結果は現状スキャンの各ステップ毎に当該 CIRASAME のスキャンされた全チャンネルの DCR が一つのファイルに書き込まれる形式となっており、チャンネル毎の threshold 値 vs DCR 値といった分かり易い形式になっていない。これは次の中間ステップを経ることでチャンネルごとの threshold 値 vs DCR 値形式に変換される。スキャンの設定 (スタート、ストップ、ステップ) は 3 章で記述したように別ファイルにストアされた設定が参照される。Transition がステイプな為、スキャンは最小ステップの 1 が推奨される。デバッグ目的など迅速に大雑把なスキャンを行いたい場合などはこの限りではない。

thresholdscan_single.py 内部で定義されている変数は以下の通り。必要に応じて適宜変更する。

```
DATA_BASEDIR    = "~/cirasame/calib/raw" # the threshold scan data base directory
CONFIG_BASEDIR  = "~/cirasame/calib/config" # the config yaml file base directory
CITIROC_PATH    = "~/cirasame/CitirocControlSoft/bin" # the CITIROC control software binary directory
```

```
HUL_PATH      = "~/cirasame/hul-common-lib/install/bin" # the HUL software binary directory
YAML_FILE     = "~/cirasame/calib/config/thresholdscan_range.yml" # the threshold scan range YA
LOG_FILE      = "~/cirasame/calib/thresholdscan_log.txt" # the threshold log file
```

スキャン終了後に元のしきい値が失われないようにスキャンの前に現在の設定ファイル(レジスター設定値)を一時メモリ上に保存し、スキャン終了後にそこから書き戻す作業をしている。元々スキャンの設定 (Begin, End, Step) は設定ファイルから読むようになっていた。スクリプトは現在 Command Line Argument から指定できるようになっている(はず)。Begin/End/Step の情報が不完全の場合は設定ファイル情報が使われる。-t オプションはファイルを作成したくないときに用いる。スキャン自体は実行される。現状ラン名のディレクトリまでは作られてしまうがその先は作成されない。Overwrite (y/n) などと Shell に聞かれたら y で ok。

用法

```
thresholdscan_single.py [-h] -r RUN -n NUMBER
                        [-b BEGIN] [-e END] [-s STEP]
                        [-t]
                        [-l {DEBUG,INFO,WARNING,ERROR,CRITICAL}]
```

optional arguments:

```
-h, --help            show this help message and exit
-r RUN, --run RUN      Run name 推奨 YYYYMMDD_HHmm
-n NUMBER, --number NUMBER
                        CIRASAME number
-b BEGIN, --begin BEGIN
                        Start value DAC 表記 (10bit) 推奨 150
-e END, --end END      End value DAC 表記 (10bit) 推奨 600
-s STEP, --step STEP   Step value 推奨 1
-t, --test            Enable test mode. スキャンは実行されるがファイルに書き込まない。
-l {DEBUG,INFO,WARNING,ERROR,CRITICAL}, --loglevel {DEBUG,INFO,WARNING,ERROR,CRITICAL}
                        Set the logging level
```

読み込みファイル

- ~/cirasame/calib/config/ipmap.yml: commonConfigReader.py ライブラリを利用して CIRASAME ボードの IP Address を CIRASAME 番号から引いてくる。そちらで参照されるファイル。
- ~/cirasame/calib/config/thresholdscan_range.yml: スキャン設定 (Begin, End, Step) を記述したファイル。yml 形式。fig/cat-thresholdscan-range.yml.png

出力ファイル

- 生データ。~/cirasame/calib/raw 以下に<run_name>/cirasameXXX というディレクトリが作られ、更にそこ以下に binary と decimal ディレクトリが作られる。各スキャン毎に binary/dataBin<DACValue>.dat 及び decimal/dataDec<DACValue>.txt というファイルが保存される。両者の内容は冗長で形式の違いのみ。以後の変換プロセスでは decimal のみが使われている。
- ログファイル。Default は~/cirasame/calib/thresholdscan_log.txt。後述するようにこのプロセスを複数同時に走らせるとスキャンデータが正常に取得できなくなることがあるのでそのデバッグ等に使う。log level は通常は default 値=WARNING でよい(はず)

例

```
~/cirasame/calib> ./bin/thresholdscan_single.py -r 20240422_0130 -n 1 -t
```

複数の CIRASAME ボードのスキャンを行う

thresholdscan_auto.sh 1 スキャンは 2024 Apr.-May のビームタイムの環境²ではおよそ 45 分掛かった³。複数の CIRASAME に対するスキャンを順番に行くと長大な時間が掛かってしまうためこれを並行して行いたいことが想定される。その場合は thresholdscan_auto.shbash script を用いる。これは内部で先述の thresholdscan_single.py を呼んでいる。これは<nparallel>でこの並行プロセス数を指定する。しかし大きな<nparallel>を指定した

²E50server03, EPYCXXX, 1Gbps network

³実は現環境での実測時間。もう少し早かったような記憶も。スキャン範囲は 150-600 の 451 ステップ。

場合 (e.g. 18) 正しいデータが取得できないケースが度々確認された。<nparallel>= 18 でも正常に終了することもあり成否に他の要因が関与していると考えられるが現状デバッグがなされていないが SiTCP の問題と考えられている。また、この並列化には gnu parallel[8] を用いているため適宜インストールしておく必要がある。本スクリプトでは argument でスキャン設定を指定する機能が実装されていない⁴。その為スキャン設定は設定ファイル (/cirasame/calib/config/thresholdscan_range.yml) にて指定する必要がある。

用法

```
thresholdscan_auto.sh <nparallel> <begin> <end> <run name>
  nparallel: the number of processes that run in parallel
  begin:     the first CIRASAME number to loop through
  end:       the last CIRASAME number to loop through
  run name:  unique run name of the scan. Recommended: YYYYDDMM_HHmm
```

読み込みファイル

- thresholdscan_single.py と同様

書き込みファイル

- thresholdscan_single.py と同様

例

```
~/cirasame/calib> ./bin/thresholdscan_auto.sh 6 1 12 20240422_0130
```

5.3.2 threshold scan raw data の整形

上で述べたように scan 結果の生データをチャンネル毎の threshold 値 vs DCR 値という分かり易い形式に変換する。入力として前ステップの生データを読み込み、出力として整形されたデータを書き出す。raw2dat.py を用いる。現状 CIRASAME ボードが 1-18 までである想定がハードコードされている。これは緊急に修正が必要。また、ラン名を指定するオプションとして -n が用いられているが、他のコマンドでは -r が用いられていることがあるのに注意。

用法

```
./raw2dat.py -n run_name
run_name: threshold scan で入れたラン名。calib/raw/(ここの名前)
```

読み込みファイル

- ~/cirasame/calib/raw/<run_name>/cirasameXXX/以下のファイル達

書き込みファイル

- ~/cirasame/calib/data/thresholdscan_YYYYMMDD_HHmm/cirasameXXX_YY.dat
(XXX=001..018, YY=01..04)

例

```
~/cirasame/calib> ./bin/raw2dat.py -n 20240422_0130
```

5.3.3 整形された threshold scan データから ROOT TGraph 形式への変換

Scan 結果のチャンネルごとの threshold 値 vs DCR 値を各々 Root の TGraph 形式に変換し Root file として保存する。この変換は現状 Root を interactive mode で立ち上げて、その Root コンソール上でマクロを実行することによって行う。Root マクロは dat2root.C である。DCR は今後の解析の都合上この段階で対数値にしている。

用法 recommended to run in a directory /cirasame/calib/ana/

```
root> .x dat2root.C(“ ../data/thresholdscan_YYYYMMDD_HHmm ”);
```

⁴TODO List

読み込みファイル

- `~/cirasame/calib/data/thresholdscan_YYYYMMDD_HHmm/cirasameXXX_YY.dat` (XXX=001..018, YY=01..04)

書き込みファイル

- `~/cirasame/calib/ana/root/thresholdscan_YYYYMMDD_HHmm.root` contains (the number of channels) TGraphs

例

```
root> .x dat2root.C(“ ../data/thresholdscan_20241106_1630 ”);
```

5.3.4 Threshold scan データの解析

前ステップで Root file に TGraph 形式で保存されたチャンネル毎の scan 結果を読み込み、解析して、チャンネル毎に Baseline, 1 p.e. の波高を DAC 値で決定し、Gain、0.5 p.e. 波高及び 3.5 p.e. 波高相当 DAC 値を推定する。Root のマクロで書かれている。threscan_ana.C を実行する。結果は pdf file および txt ファイルで出力される。

数千チャンネルの解析を自動化して行うことになるが、バイアス電圧の設定などによりスキャン結果が変化するため手で解析スクリプトのパラメーターを調整する必要があることが想定される。そのためスクリプトが行っている解析アルゴリズムを図 5.3 に簡単に解説する。

用法 recommended to run in a directory `~/cirasame/calib/ana/`

```
root> .x threscan_ana_newfit.C(“ ../root/thresholdscan_YYYYMMDD_HHmm.root ”);
```

読み込みファイル

- `~/cirasame/calib/ana/root/thresholdscan_YYYYMMDD_HHmm.root`

書き込みファイル

- 結果を視覚的に確認するためのグラフをまとめた pdf ファイル⁵。図 5.4 例えば `~/cirasame/calib/ana/pic/thresholdscan_YYYYMMDD_HHmm.pdf`。ここに全チャンネル数個別のプロット、ASIC 毎のプロット、CIRASAME 毎のプロットが含まれる。出力例が図 5.4。
- 結果をテキスト形式でまとめたもの⁶。例えば `~/cirasame/calib/ana/out/thresholdscan_YYYYMMDD_HHmm.out`
At this moment this file is created in a directory `/cirasame/calib/ana/`. Move it to out directory manually. 解析の結果見積もられた 3.5 p.e. 相当のしきい値が書き込まれる。また、ゲイン情報は次ステップで較正に用いられる。

例

```
root> .x threscan_ana_newfit.C(“ ../root/thresholdscan_20241106_1630.root ”);
```

5.4 Threshold scan データ解析結果を元に CIRASAME 毎の較正を行う

5.6 章で定義されたゲイン較正のステップ 1 で実行される。vcommon_adjuster.py を用いる。説明加筆ステップ 1 のあとは apply_bias.py 4.1.1 章参照。

用法

```
python3 vcommon_adjuster.py -o <outfilename> -x <xlsxfilename> --begin <begin CIRASAME ID> --  
-o / -outfile: The name of ~_chan.out file from your second scan. Place the file in ana/out d  
-x /--xlsx : The name of your setting file. Place the file in xlsx directory.  
--begin : Write the ID of 1st CIRASAME. Default is 1.  
--end : Write the ID of last CIRASAME. Default is 18.
```

⁵チャンネル毎の DCR とヒストグラムのプロット (図 5.3) についての説明を追記する必要あり

⁶fitting がうまくいかなかった Channel が含まれる項目は nan で埋められる。この nan の vcommon_adjuster.py および vad-just_adjuster.py での処理は以降の todo

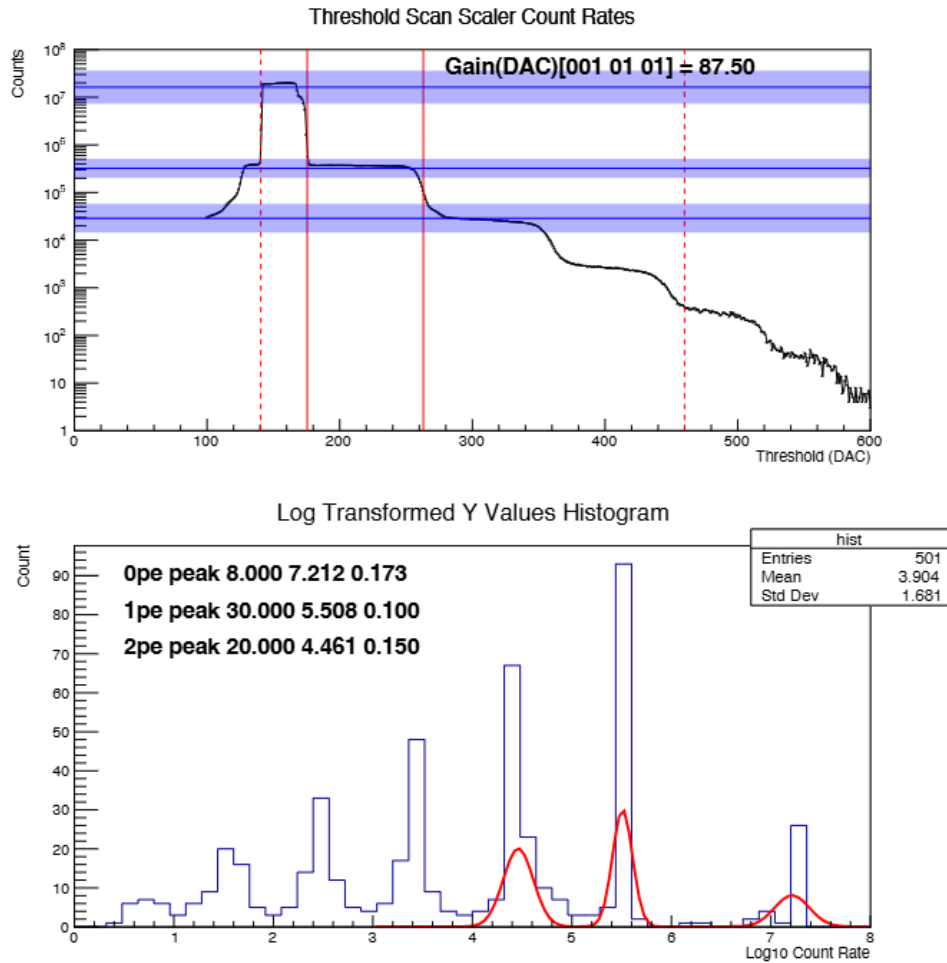


Figure 5.3: Threshold scan の一例。上図を横に投射した時のヒストグラムが下図になる。ゲインを算出するためにまず下図のヒストグラムの横軸の値の大きい方から3つ分のピークを取り出す。Gaussian でフィットしており、その時の mean と sigma が上図の青線、および半透明の青い領域になる。次にそれぞれの青い範囲に入る最初と最後のデータ点を求め、隣り合う領域の境界をそれぞれの領域の最初と最後の点の平均で求める (赤い縦実線)。この2本の線がそれぞれノイズ・1光子信号の波高に相当するのでこれらの差をゲインとして計算している。

読み込みファイル

- `~/cirasame/calib/ana/out/threscan_ana_crsim_YYYYMMDD_HHmm.out`
- `~/cirasame/calib/ana/out/threscan_ana_asic_YYYYMMDD_HHmm.out`
- `~/cirasame/calib/ana/out/threscan_ana_chan_YYYYMMDD_HHmm.out`

出力ファイル `~/cirasame/calib/xlsx/cirasame_setting.xlsx`

例

```
~/cirasame/calib/bin> ./vcommon_adjuster.py ../ana/out/threscan_ana_crsim_20240423_2000.out -o ../xlsx/ci
```

5.5 Threshold scan データ解析結果を元にチャンネル毎の較正を行う

5.6 章で定義されたゲイン較正のステップ2で実行される。`vadjust_adjuster.py` を用いる⁷。較正の精度を上げるためにイタレーションを行う際にも用いる。`xls2yaml.py`(4.1.2 章) `register2cirasame.py`(4.1.2 章)

⁷`vadjust_adjuster` は`***.chan.***.out`を読み、その平均値を求めてシフト分を計算するが、これを全体ではなく CIRASAME 毎に行うやつも欲しい (つまり 128 チャンネル分の平均値からシフトを計算するもの)

用法

```
python3 vadjust_adjuster.py -o <outfilename> -x <xlsxfilename> --begin <begin CIRASAME ID> --  
-o / -outfile: The name of ~_chan.out file from your second scan. Place the file in ana/out d  
-x /--xlsx: The name of your setting file. Place the file in xlsx directory  
--begin      : Write the ID of 1st CIRASAME. Default is 1.  
--end        : Write the ID of last CIRASAME. Default is 18.
```

読み込みファイル

- ~/cirasame/calib/ana/out/threscan_ana_crsm_YYYYMMDD_HH:mm.out
- ~/cirasame/calib/ana/out/threscan_ana_asic_YYYYMMDD_HH:mm.out
- ~/cirasame/calib/ana/out/threscan_ana_chan_YYYYMMDD_HH:mm.out

出力ファイル /cirasame/calib/xlsx/cirasame_setting.xlsx

例

```
~cirasame/calib/bin> ./vadjust_adjuster.py ../ana/out/threscan_ana_chan_20240423_2000.out -o
```

5.6 Threshold scan データ解析からしきい値を反映させる

Threshold scan で見積もられた見積もられたゲインを元に決定された 3.5 p.e. 相当のしきい値をスプレッドシートに書き込む。通常はゲイン較正が完了した段階で実行するが、原理的には章で定義されたゲイン較正ステップのどの段階で実行しても良い。その際はゲインのばらつきを反映してしきい値になっている。out2xls.py を用いる。

用法

```
out2xls.py [-h] [-o OUTPUT] input_file  
positional arguments:  
  input_file      Path to the input file  
optional arguments:  
  -h, --help      show this help message and exit  
  -o OUTPUT, --output OUTPUT  
                  Path to the output Excel file (default: output.xlsx)
```

読み込みファイル

- ~/cirasame/calib/ana/out/threscan_ana_crsm_YYYYMMDD_HH:mm.out
- ~/cirasame/calib/ana/out/threscan_ana_asic_YYYYMMDD_HH:mm.out ??
- ~/cirasame/calib/ana/out/threscan_ana_chan_YYYYMMDD_HH:mm.out

出力ファイル

- ~/cirasame/calib/xlsx/cirasame_setting.xlsx

例

```
~cirasame/calib/bin> ./out2xls.py ../ana/out/threscan_ana_chan_20240423_2000.out  
-o ../xlsx/cirasame_setting.xlsx
```

5.7 較正された CIRASAME 設定を設定ファイルに反映させる

Threshold scan データ解析結果を元に校正された CIRASAME 設定 (xlsx) を下位ソフトウェア用の設定ファイル (yaml) に反映させる。xls2yaml.py を用いる。用法は 4.1 章参照のこと。

5.7.1 CIRASAME 設定ファイルを CIRASAME に送り反映させる

register2cirasame.sh を用いる。用法は 4.1 章参照のこと。

例

実際にはマニュアルを順番に見ながら作業すると思われるので、ページを行ったり来たりしなくてもよいように例だけは書いておく。

5.8 実際のゲイン較正の手順の例まとめ

1. 初期設定
 - (a) 初期 xls ファイルの準備
 - (b) xls2yaml.py
 - (c) apply_bias.py # V_{common} の反映
 - (d) register2cirasame.py # V_{adjust} の反映
2. スキャン一回目: V_{common} の調整
 - (a) Threshold Scan⁸
 - i. thresholdscan_auto.C()
 - ii. raw2dat.py
 - iii. dat2root.C()
 - iv. threscan_ana.C()
 - (b) vcommon_adjuster.py # V_{common} 較正值計算
 - (c) apply_bias.py # V_{common} の反映
3. スキャン二回目: V_{adjust} の調整
 - (a) Threshold Scan
 - i. thresholdscan_auto.C()
 - ii. raw2dat.py
 - iii. dat2root.C()
 - iv. threscan_ana.C()
 - (b) vadjust_adjuster.py # V_{adjust} 較正值計算
 - (c) xls2yaml.py
 - (d) register2cirasame.py # V_{adjust} の反映
4. スキャン三回目: V_{adjust} の再調整 (推奨だが必須ではない)
 - (a) Threshold Scan
 - i. thresholdscan_auto.C()
 - ii. raw2dat.py
 - iii. dat2root.C()
 - iv. threscan_ana.C()
 - (b) vadjust_adjuster.py # V_{adjust} 較正值計算
 - (c) xls2yaml.py
 - (d) register2cirasame.py # V_{adjust} の反映
5. 結果の確認
 - (a) Threshold Scan
 - i. thresholdscan_auto.C()
 - ii. raw2dat.py
 - iii. dat2root.C()
 - iv. threscan_ana.C()
 - (b) out2xls.py #しきい値の反映
 - (c) register2cirasame.py #しきい値の反映

⁸Threshold Scan の 4 手順をまとめたスクリプトを一度作ったがこの間どうも失われてた。

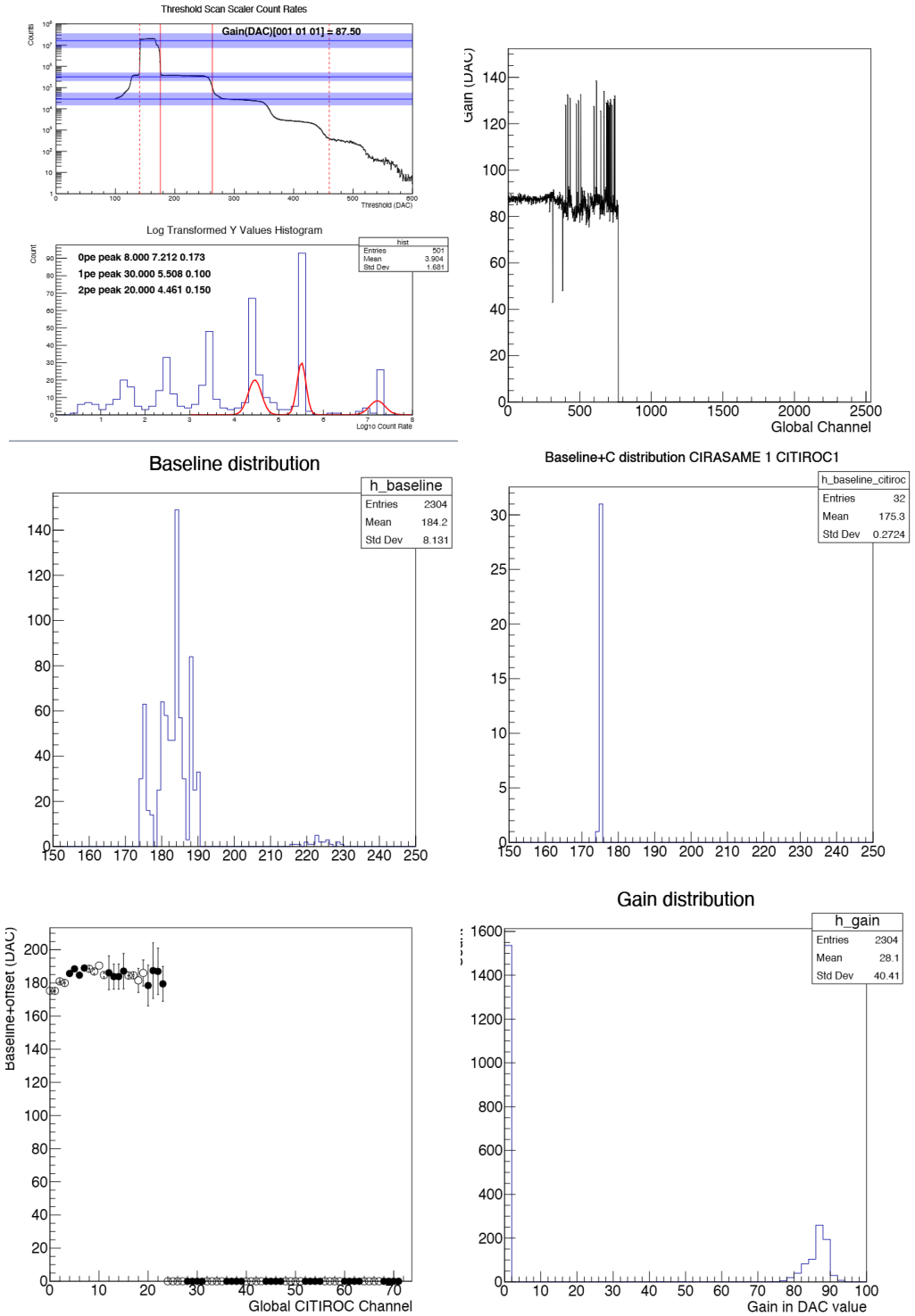


Figure 5.4: `threscan_ana.C` 出力の例 (全チャンネル入っているベターなもの置き換える)。左上のパネルは図 5.3 と同じ。左下のパネル。フルと白抜きが混ざっているのは CITIROC の塊を視認しやすくするため。

Chapter 6

Temperature Monitoring

CIRASAME ボード上の FPGA 温度センサーをもちいて、擬似的に SiPM の温度をモニターする。ファイルは `~/cirasame/temperature_monitor/` 以下。

`temperature_scan.sh`
is a standalone script that checks the temperatures of CIRASAME
input: `temperature_scan.in`
output: STDOUT
`~/cirasame/temperature_monitor/> ./temperature_scan.sh`

`temperature_scan.in`
is a parameter file to be used by the temperature scan/monitor/plotter suit.
It contains a list of IP addresses of CIRASAMEs to be scanned.

`temperature_monitor.sh`
runs the `temperature_scan.sh` in a regular interval (e.g. 1 min) and logs into a file. This has to be run on
input: `temperature_scan.in`
is a parameter file to be used by the temperature scan/monitor/plotter suit.
It contains a list of IP addresses of CIRASAMEs to be scanned.
output: `temperature_scan.dat`
format
datetime(ISO8601) temp1 temp2 temp3 ... tempN
e.g.:
`~/cirasame/temperature_monitor/> ./temperature_monitor.sh`

`temperature_plotter.py`
a python script to make a plot. Run on another machine than the daq server, e50server03. It is tested to run on
`~/cirasame/temperature_monitor/> ./temperature_plotter.py`
input: `temperature_scan.dat`
input: `temperature_scan.in`
output: `pic/temperature_plotterXXXXXX.png`

`temperature_publisher.py`
publish the plot on the web using Flask.
It's running on nlabdaq6 now.
`~/cirasame/temperature_monitor/publisher> ./temperature_publisher.py`
input: `temperature_plotter_XXXXXX.png`

to view the CIRASAME temperature trend, Do `http://nlabdaq6:5000/` from JLAN network.

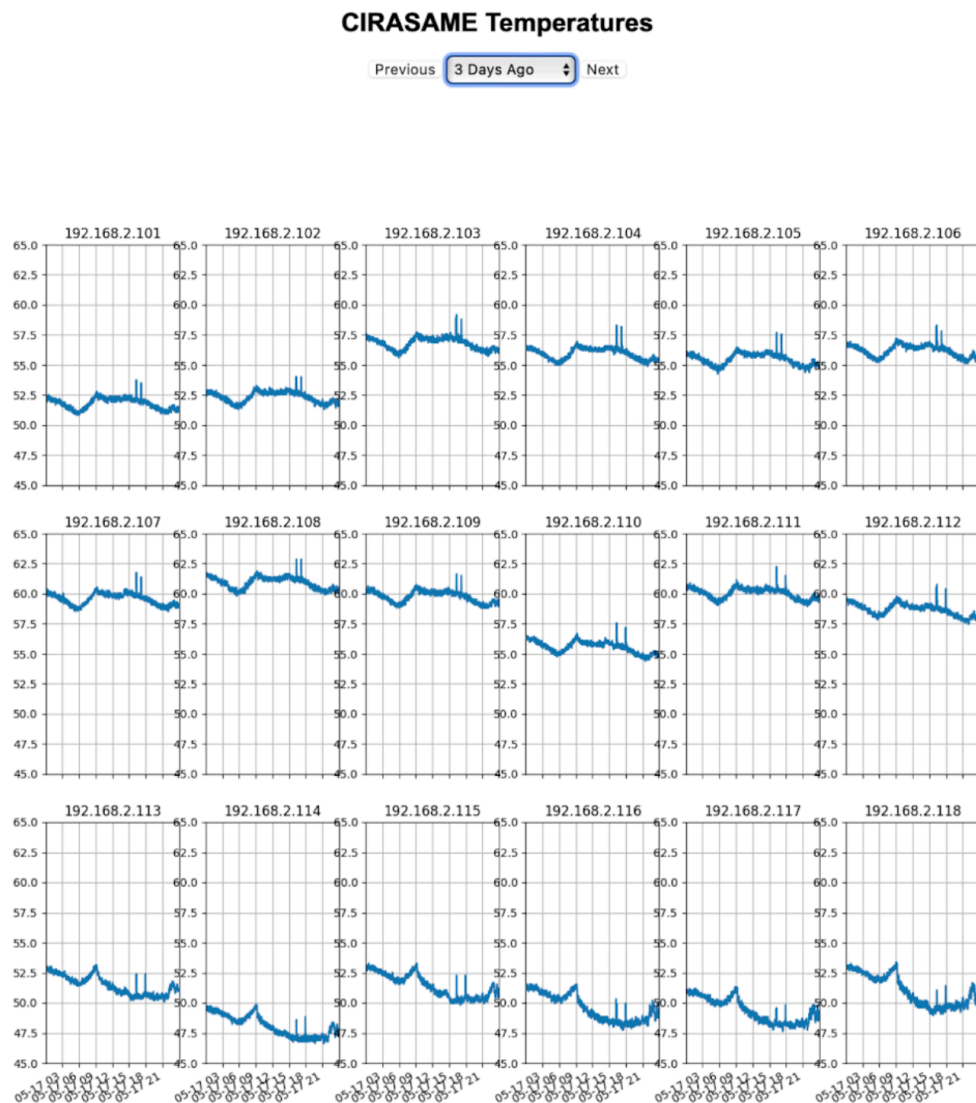


Figure 6.1: temperature.publisher.py http サーバーの出力例

List of Figures

2.1	CIRASAME の画像	4
2.2	CIRASAME への電力供給	5
2.3	CIRASAME 上の DIP スイッチ拡大図	5
2.4	CITIROC の回路図	6
2.5	実際に SiPM に印加される電圧は CIRASAME ボードの機能で提供される電圧 V_{common} (40-70 V) 及び CITIROC チップの提供するチャンネルごとの微調整用電圧 V_{adjust} (0-2.5 V) の 2 弾構成になっている。XXX	6
3.1	<code>~/cirasame/calib/config</code> ディレクトリ及び <code>~/cirasame/calib/config/cirasame001</code> ディレクトリ内のファイル構成例。	8
3.2	<code>~/cirasame/calib/config/cirasame001/DiscriMask.yml</code> ファイルの例。	9
3.3	<code>~/cirasame/calib/config/cirasame001/InputDAC.yml</code> ファイルの例。	9
3.4	<code>~/cirasame/calib/</code> 以下のディレクトリの概要。リストが長くなりすぎないように <code>ana/log</code> , <code>ana/out</code> , <code>ana/pic</code> , <code>ana/root</code> , <code>config</code> , <code>data</code> , <code>raw</code> , <code>xls</code> ディレクトリの中身は適当に短くしてある。整理しきれていない、すでに使われていないと思われるスクリプトなどもあるので注意。	12
3.5	<code>cirasame_settings.xlsx</code> の例。複数ボードで組まれシステム全体に関する設定内容 (上) と各ボード毎の設定内容 (下) からなる。これらがタブとして一つのスプレッドシートに纏めてある。**(ここに CIRASAME ボードの個体認識が ip address 基準で行われていて、該当 IP を消すとスキップすることを記述する)	14
3.6	<code>~/cirasame/calib/config/thresholdscan-range.yml</code> ファイルの例。	14
5.1	ゲイン調整各ステップでのゲイン分布ヒストグラム (上) とゲイン vs Global Channel の図 (下)。各々左から順番に 1) 全ての channel に一定の V_{bias} がかかっている初期状態 2) CIRASAME ボード毎に V_{common} の調整を行った状態 3) Channel 毎に V_{bias} を調整した状態 4) 2 から 3 のステップをイタレートした状態。ゲイン分布の RMS は各々 7.3, 3.6, 1.5, 1.0 で、最終的に 1.1% の均一度が得られている。	19
5.2	Threshold scan の例 (下) 及び SiPM の波高分布 (上) の対応。は高分布は SiPM のオシロスコープの出力を Persistency On で取得したものを 90 度回転させてある。下図、縦軸は DCR [Hz]、横軸は CITIROC Discriminator のしきい値の DAC 値。しきい値が p.e. 毎の波高を超えるごとに階段状の構造が現れる。この例では 5 p.e. 程度のプラトーまで見えている。DCR 測定時間は各々 1 秒。本例の DAC と実際のしきい値の関係は $V = -0.114 \times \text{DAC} + 70.567$ 。	21
5.3	Threshold scan の一例。上図を横に投射した時のヒストグラムが下図になる。ゲインを算出するためにまず下図のヒストグラムの横軸の値の大きい方から 3 つ分のピークを取り出す。Gaussian でフィットしており、その時の mean と sigma が上図の青線、および半透明の青い領域になる。次にそれぞれの青い範囲に入る最初と最後のデータ点を求め、隣り合う領域の境界をそれぞれの領域の最初と最後の点の平均で求める (赤い縦実線)。この 2 本の線がそれぞれノイズ・1 光子信号の波高に相当するのでこれらの差をゲインとして計算している。	25
5.4	<code>threscan_ana.C</code> 出力の例 (全チャンネル入っているベターなものと置き換える)。左上のパネルは図 5.3 と同じ。左下のパネル。フルと白抜きが混ざっているのは CITIROC の塊を視認しやすくするため。	28
6.1	<code>temperature_publisher.py</code> http サーバーの出力例	30

Listings

1 ~/cirasame/calib/config/RegisterValue.yml ファイルの例。102 ~/cirasame/calib/config/settings.yml ファイルの例。103 ~/cirasame/calib/config/ipmap.yml ファイル例。134 ~/cirasame/calib/config/commonpath.yml ファイルの例。135 read_scr を用いたスケーラーの読み出し方 206 スケーラーの出力情報を 10 進数で表示 207 10 進数へ変換後のスケーラーデータ (CIRASAME のファームウェアが変更されたときはこのフォーマットが変更されていることが偶にあるため注意！)208 write_register を用いて全チャンネルの scaler 値を 0 にする 20

Bibliography

- [1] J. Fleury, et al. Journal of Instrumentation, 9 (2014), p. C01049
- [2] hoge et al. hage
- [3] R. Honda et al. <https://spadi-alliance.github.io/ug-cirasame/>
- [4] hoge et al. CIRASAME
- [5] hoge et al. NestDAQ
- [6] Shirotori et al. J-PARC T103
- [7] R. Honda et al. *Algebra*, volume 211 of *Graduate Texts in Mathematics*. Springer-Verlag, New York, third edition, 2002.
- [8] Gnu Parallel <https://www.gnu.org/software/parallel/>